Кто о чём пишет: краткий пересказ НИРС

Полыга, Ковалев, Перушкин, Ширинкин. Для каждого: о чём работа простыми словами, что сделано в первой главе, что предлагается дальше. В конце справочник по сложным технологиям и терминам, которые в работах упоминаются без объяснений.

Подготовлено 24.09.2026 по последним версиям файлов в репозитории Diplom_2026.

Автор	Тема	По какому файлу пересказано
Полыга К.Ю.	Оценка и регулирование киберрисков инфраструктуры контейнеризации Kubernetes при компрометации контейнерных приложений	`ВКР-Полыга/НИР-Полыга/Полыга К.Ю. ВКР 23.09.26.pdf`
Ковалев И.М.	Регулирование рисков корпоративной системы управления доступом при компрометации учётных записей	`ВКР-Ковалев/Main/ Диплом_АКТУАЛ_23.09.pdf`
Перушкин Н.П.	Оценка киберрисков серверной инфраструктуры Linux в условиях эксплуатации ошибок конфигурирования сервисов	`ВКР-Перушкин/ ВКР_Перушкин_Глава1_v6.pdf`
Ширинкин С.И.	Оценка и регулирование киберрисков корпоративной DevOps-инфраструктуры при нарушении безопасности CI/CD-процессов	`ВКР-Ширинкин/НИРС Ширинкин.pdf`

Содержание

1. Общая схема: что объединяет все четыре работы
2. Полыга: Kubernetes
3. Ковалев: учётные записи и управление доступом
4. Перушкин: серверы на Linux
5. Ширинкин: CI/CD-конвейер
6. Справочник А. Контейнеры и Kubernetes (со схемой)
7. Справочник Б. Сервер на Linux и сканеры
8. Справочник В. Корпоративный доступ: каталог, Kerberos, MFA, IdM/PAM
9. Справочник Г. DevOps и CI/CD (со схемой)
10. Справочник Д. Риск-анализ: CVE, CVSS, EPSS, ATT&CK, FAIR, ROSI
11. Справочник Е. Нормативка: какие документы упоминаются

1. Общая схема: что объединяет все четыре работы

Все четыре НИРС написаны по одной методике кафедры (методика Остапенко), поэтому у них одинаковый каркас. Разобравшись в нём один раз, любую из работ читать легко.

Общая идея всех четырёх работ в двух фразах

Главную угрозу для каждого объекта создают не «дырки» в программном коде (у таких есть номер CVE и готовая оценка опасности), а **неправильные настройки, лишние права и слабая организация процессов**. У этих проблем нет номера CVE, поэтому существующие калькуляторы опасности (CVSS, EPSS) их не видят, а сканеры дают только «прошло / не прошло». Каждый автор предлагает **интеллектуальную систему** (то есть с блоком машинного обучения), которая переводит такие недостатки в **число риска заранее, до атаки**, подбирает меры защиты и доказывает их выгоду.

Как устроена логика («проактивное управление риском»)



Рисунок 1. Общий цикл, по которому построены все четыре работы

Каркас каждой работы

- **Введение.** Статистика, почему тема важна; **противоречия** («есть вот это, а нужно вот то»); объект, предмет, цель; **задачи** (по четыре, у Ширинкина по шесть); новизна и практическая ценность.
- **Глава 1** (она и составляет НИРС): исследование объекта, статистики, нормативки, существующих средств и методов оценки риска, затем обоснование противоречий и задач.
- **Главы 2–5** (будут в ВКР): 2 — информационное обеспечение (база данных недостатков и угроз); 3 — оценка риска (ML + FAIR); 4 — регулирование риска (подбор мер, остаточный риск); 5 — программа-автоматизация и расчёт эффективности (ALE, ROSI).

Базовая формула везде одна: риск = вероятность ущерба × величина ущерба (**R = P · U**). Что такое FAIR, ALE, ROSI и прочие сокращения, объяснено в справочнике Д.

Четыре работы рядом

	Полыга	Ковалев	Перушкин	Ширинкин
Что защищаем	Кластер Kubernetes	Систему управления доступом (каталог, учётки, роли)	Серверы на Linux и их сервисы	Конвейер сборки и поставки ПО (CI/CD)
Для чего считаем риск	Для каждой рабочей нагрузки (пода)	Для каждой учётной записи	Для узла и сервиса, для цепочки атаки	Для этапа конвейера, секрета, артефакта
Главный недостаток без CVE	Опасные настройки кластера: привилегированный под, каталог узла внутри контейнера, лишние права RBAC	Накопленные лишние права, слабая аутентификация	Ошибки конфигурирования сервисов (плюс учёт обычных CVE)	Плохая организация конвейера: вечные токены, нет проверки подписи, лишние права сборщика

Роль ML	Предсказывает уровень риска нагрузки по её манифесту	Isolation Forest ищет аномальное накопление прав (без учителя)	Предсказывает критичность и вероятность эксплуатации для ошибок конфигурации (с учителем)	Классифицирует конвейеры по уровню риска
С чем сравнивается	kube-bench, Kubescape, Trivy, Falco	IdM, IGA, PAM, аудит AD, UEBA, SIEM	OpenSCAP, Lynis, CIS-CAT, Wazuh, ScanOVAL, Nessus, MaxPatrol VM	Checkov, Snyk, GitGuardian, Trivy, SLSA/in-toto, Sigstore

2. Полыга: риски Kubernetes при взломе контейнерного приложения

«Разработка интеллектуальной системы оценки и регулирования киберрисков инфраструктуры контейнеризации Kubernetes при компрометации контейнерных приложений»

> **Если совсем коротко**
>
> Современные приложения работают в контейнерах, а контейнерами управляет Kubernetes. Если хакер взломал одно приложение (один под), дальше всё зависит от того, **как этот под настроен**. Запущен в привилегированном режиме, видит папки сервера или у него «лишние» права в кластере: тогда из одного приложения можно выбраться на сервер (узел), а оттуда захватить управление всем кластером. Эти настройки видны заранее в конфигурации, значит риск можно посчитать до атаки. Работа про систему, которая это делает.

Проблема и цифры

- Kubernetes фактически стандарт: в промышленной эксплуатации его используют 66 % организаций (CNCF).
- 86–93 % организаций ежегодно (2022–2025) сталкиваются хотя бы с одним инцидентом в контейнерной среде.
- Главная причина инцидентов не уязвимость образа (27–33 %), а **ошибка конфигурации (45–53 %)**.
- Доля сервисных аккаунтов с правами, которые они вообще не используют, выросла с 84 % до **98 %**; около трети нагрузок работает с правами root.
- В банке угроз ФСТЭК из 222 угроз к виртуализации и контейнеризации применимы 24.

Что сделано в главе 1

- **1.1 Объект защиты.** Разобрано, как устроен кластер: плоскость управления (kube-apiserver, etcd, scheduler, controller-manager), узлы (kubelet, kube-proxy, среда исполнения), поды, сервисные аккаунты и RBAC, реестр образов. Таблица «компонент → что будет, если его взломают». Выделены четыре **границы доверия**: контейнер ↔ узел, узел ↔ плоскость управления, между пространствами имён, кластер ↔ внешний реестр. Главный вывод: *ущерб от взлома пода определяется не его кодом, а тем, до чего из него можно дотянуться*. Два одинаковых приложения дают разный ущерб, если одно привилегированное, а другое нет.
- **1.2 Статистика** за 2022–2025 (таблица): инциденты ожидаемы, конфигурация стабильно главнее уязвимостей, лишние права почти у всех.
- **1.3 Нормативка.** Приказы ФСТЭК № 17/21/239, ГОСТ Р 56938 (виртуализация), ГОСТ Р 56545/56546 (уязвимости), ISO 27005/31000. Таблица соответствия «мера ФСТЭК → какой параметр кластера её реализует» (например, УПД → привязки RBAC, защита среды виртуализации → запрет privileged и hostPath). Профили NIST SP 800-190 и CIS Kubernetes Benchmark описывают риски качественно, чисел не дают.
- **1.4 Средства.** kube-bench, Kubescape, Trivy, Falco выдают «пройдено / не пройдено» и уровни критичности, но: нет меры риска, не учитывают сочетание недостатков (привилегированный под *плюс* лишняя роль), не обосновывают выбор мер. Годятся как источник исходных данных.
- **1.5 Калькуляторы риска.** CVSS 3.1 и 4.0, EPSS, FAIR, ALE и ROSI. Вместе они покрывают весь цикл, но их надо «пересадить» с уязвимостей на недостатки конфигурации кластера.
- **1.6 Противоречия и задачи** (4 штуки, по одной на главы 2–5).

Что предлагается в ВКР

- **Глава 2:** единая база: активы кластера, границы доверия, недостатки конфигурации, связанные с техниками атак (MITRE ATT&CK for Containers) и ущербом.
- **Глава 3:** ML-модель оценивает уровень риска рабочей нагрузки **по параметрам её манифеста**; сценарии атаки «выход из контейнера → узел → плоскость управления» строятся как пути в графе; первичный риск считается по FAIR.
- **Глава 4:** автоматический выбор мер защиты с учётом масштабности и профильности, расчёт остаточного риска.
- **Глава 5:** программа, прогоняющая весь цикл, и проверка эффективности мер через ALE и ROSI.

Непонятные слова (под, нода, RBAC, hostPath, etcd…) разобраны в справочнике А со схемой кластера.

3. Ковалев: риски учётных записей в корпоративной системе доступа

«Разработка интеллектуальной системы регулирования рисков корпоративной системы управления доступом при компрометации учётных записей»

Если совсем коротко

Хакеру не нужно ничего взламывать, если у него есть логин и пароль сотрудника: система сама пустит его со всеми правами этой учётки, и отличить его от настоящего сотрудника почти невозможно. Значит, ущерб определяется тем, **сколько прав накопила учётная запись**. А права копятся годами: при переводах, замещениях, проектах их выдают быстро и почти никогда не забирают. Работа про систему, которая для каждой учётки считает вероятность взлома и возможный ущерб, находит «аномально разросшиеся» учётки и подсказывает, что урезать в первую очередь.

Проблема и цифры

- Verizon DBIR 2025: **22 %** нарушений начинались с украденных учётных данных, человеческий фактор присутствовал в 60 %.
- Positive Technologies: 55 % российских организаций за год столкнулись с атаками; на пентестах в **96 %** случаев получают полный доступ ко всей инфраструктуре, причём внутри периметра двигаются в основном под легитимными учётками.
- Накопление прав системное: заявка на выдачу прав есть, заявки на отзыв нет; аттестация формальная; временные права выдаются бессрочно.

Что сделано в главе 1

- **1.1 Объект.** Система доступа = служба каталога + аутентификация + авторизация + управление жизненным циклом (IdM/IGA) + привилегированный доступ (PAM) + аудит. Типы учёток и их риск: пользовательские, административные, **сервисные** (пароль годами лежит в конфигах, MFA к ним неприменим, владельца нет), технологические, внешние (подрядчики, хуже всего контролируются). Модели доступа DAC, MAC, **RBAC** (выбрана основной), ABAC. Описана «эрозия» ролевой модели: права в обход ролей, вложенные группы, непрекращённые проектные права, делегирование. Вывод: считать надо **фактические** права из каталога, а не те, что записаны на бумаге. Введены группы признаков учётки: права, аутентификация, активность, отклонение от коллег по должности, ценность доступных ресурсов.
- **1.2 Нормативка.** 149-ФЗ, 152-ФЗ, приказы ФСТЭК 17/21/239 (группы мер ИАФ и УПД, таблица «мера → как отражается в риск-анализе»), ГОСТ Р 57580.1 (финансы), ГОСТ Р 58833, NIST SP 800-53/800-30. Вывод: требования адресованы *процессу* («пересмотр прав проводится»), а не конкретной учётке, и чисел не дают.
- **1.3 Уровни аутентификации NIST (AAL1–AAL3) и MFA.** Даже MFA бывает разной: SMS-коды и push-подтверждения (AAL2) ломаются фишингом в реальном времени и «утомлением подтверждениями». Разобраны атаки на протоколы: передача хэша (NTLM), подбор паролей сервисных учёток по билетам (Kerberoasting), «золотой билет», перечисление каталога. Все они используют штатные механизмы, CVE у них нет. SSO усиливает последствия кражи сессии. Уровень AAL учётки станет основой шкалы вероятности взлома.
- **1.4 Средства.** IdM/IGA, PAM, аудит каталогов, UEBA, SIEM: ни одно не даёт числа риска на учётку; поведенческий анализ вообще не видит тихого накопления прав.

- **1.5 Модели риска.** ISO/NIST/OCTAVE опираются на экспертные шкалы; CVSS/EPSS неприменимы (у учётки нет CVE); лучше всего подходит **FAIR**: R = P·U, частота потерь LEF = TEF · V. Для поиска лишних прав выбран **изолирующий лес (Isolation Forest)**: не нужна размеченная выборка, быстрый и, главное, объяснимый (руководителю надо понятно обосновать, почему урезаем права).
- **1.6 Противоречия и задачи** (4 штуки).

Что предлагается в ВКР

- **Глава 2:** база: компоненты системы доступа, свойства учёток и ролей, сценарии компрометации (тактики Credential Access и Persistence из ATT&CK), синтетический каталог учёток для экспериментов.
- **Глава 3:** шкала вероятности по уровню аутентификации, Isolation Forest для аномального накопления прав, первичный риск по FAIR, **риск-ландшафт** учёток (кто опаснее всех).
- **Глава 4:** подбор мер (урезать права, усилить аутентификацию и т. д.), остаточный риск.
- **Глава 5:** программа и расчёт ROSI.

> **Осторожно, путаница с «PAM»**
>
> У Ковалева PAM означает **Privileged Access Management** (управление привилегированным доступом). У Перушкина в Linux встречается **Linux-PAM** (подключаемые модули аутентификации). Это две разные вещи с одинаковой аббревиатурой.

4. Перушкин: риски Linux-серверов из-за ошибок настройки

«Разработка интеллектуальной системы оценки киберрисков серверной инфраструктуры Linux в условиях эксплуатации ошибок конфигурирования сервисов»

> **Если совсем коротко**
>
> Одна и та же программа одной и той же версии на сервере может быть либо безопасной, либо открытой дверью. Всё зависит от настроек: слушает ли она только сам сервер или весь интернет, спрашивает ли пароль, от какого пользователя запущена. Известные уязвимости (CVE) и ошибки настройки работают **в паре**: настройка открывает доступ к уязвимости, уязвимость даёт root. Сегодня их проверяют разные программы по отдельности и выдают списки «соответствует / не соответствует». Работа про систему, которая оценивает их **вместе**, строит цепочки атаки и считает риск.

Проблема и цифры

- Linux занимает 62,5 % веб-серверов с известной ОС (Unix-подобные в целом 92,1 %); плюс импортозамещение: госсектор и КИИ переходят на отечественные Linux (Указ № 166, Astra Linux).
- BI.ZONE: около **90 %** организаций нарушают хотя бы одно требование ФСТЭК по безопасной настройке Linux, 48 % нарушают четыре и более, полностью выполняют лишь 8 %.
- Positive Technologies: в 44 % инцидентов вход был через уязвимые веб-приложения на периметре; атаки через подрядчиков 15 %. InfoWatch: за 2023–2025 утекло более 4,5 млрд записей персональных данных.

Что сделано в главе 1

- **1.1 Объект.** Инфраструктура разбита на 8 групп сервисов: G1 удалённый доступ (SSH), G2 аутентификация (PAM, пароли), G3 файлы и права (SUID, sudo), G4 сетевые сервисы (веб, СУБД), G5 изоляция (мандатный контроль, контейнеры), G6 планировщик (cron), G7 журналирование, G8 пакеты и обновления. Введена формальная модель: узлы, сервисы, «снимок конфигурации», активы, достижимость. Перечислены наблюдаемые признаки для ML (доступность, аутентификация, полномочия, изоляция, журналы, состав ПО, ценность активов).
- **1.2 Ошибка конфигурации против уязвимости кода.** Ошибка конфигурации не привязана к версии, не имеет CVE, исправляется правкой параметра и может вернуться (**конфигурационный дрейф**). Четыре формы их связи с CVE (таблица с примерами):
  - настройка **открывает доступ** к уязвимости: Redis CVE-2022-0543 опасен, только если Redis доступен без пароля со всех интерфейсов;
  - настройка **определяет последствия**: сервис от root или в привилегированном контейнере даёт больше;
  - **звенья одной цепочки**: подбор пароля по SSH без ограничения попыток, затем pkexec CVE-2021-4034 даёт root;
  - настройка **вместо патча**: для OpenSSH CVE-2024-6387 временно помогало изменить `LoginGraceTime`.
- **1.3 Дерево угроз.** Классы недостатков: доступность, аутентификация, полномочия, изоляция, криптография, наблюдаемость, доверие к источникам ПО. Нарушители по методике ФСТЭК. Риски технические, информационные, экономические, правовые. Листья дерева станут строками базы (гл. 2) и получат числа риска (гл. 3). Отмечено, что дерево не показывает цепочки, для них нужен граф.

- **1.4 Нормативка.** Законы, приказы ФСТЭК, методический документ ФСТЭК по безопасной настройке Linux, методика оценки критичности уязвимостей и руководство по управлению уязвимостями ФСТЭК, CIS Benchmarks, DISA STIG. Вывод: требования есть, но они бинарные («есть мера / нет меры») и не позволяют сравнить недостатки по риску.
- **1.5 Средства.** OpenSCAP, Lynis, CIS-CAT, Wazuh, ScanOVAL, Nessus, MaxPatrol VM: конфиги и CVE проверяются отдельно, никто не считает риск и не видит цепочек.
- **1.6 Методы оценки.** CVSS 4.0 умеет учитывать окружение (сервис на localhost: вектор атаки локальный, а не сетевой), но как выводить эти метрики из настроек, не прописано. EPSS и CISA KEV работают только для CVE. FAIR переводит всё в деньги.
- **1.7 Графы атак.** Дерево атак (Шнайер), автоматические графы (Sheyner), логический анализатор MulVAL (уже соединяет уязвимости и конфигурацию, но конфигурацию видит как фиксированные факты). Привязка шагов к CAPEC и ATT&CK.

Что предлагается в ВКР

- **Глава 2:** база «CVE ↔ ошибки конфигурации, которые делают её эксплуатируемой ↔ техники ATT&CK ↔ ущерб».
- **Глава 3:** пересчёт CVSS 4.0 по фактическим настройкам сервера; ML-модель (с учителем) предсказывает критичность и вероятность эксплуатации для ошибок конфигурации (по аналогии с CVE тех же типов CWE); всё объединяется в граф атак, первичный риск по FAIR.
- **Глава 4:** для каждого узла графа сравнивается **обновить ПО или перенастроить сервис**; очередь исправлений по вкладу в сценарии атак.
- **Глава 5:** программа снимает конфигурацию и список пакетов, пересчитывает риск при каждом изменении, проверяет эффективность мер.

5. Ширинкин: риски конвейера сборки и поставки (CI/CD)

«Разработка интеллектуальной системы оценки и регулирования киберрисков корпоративной DevOps-инфраструктуры при нарушении безопасности CI/CD-процессов»

> **Если совсем коротко**
>
> Код программы проходит «конвейер»: репозиторий → автоматическая сборка → подключение чужих библиотек → хранилище готовых сборок → выкладка на боевые серверы. Конвейеру доверяют безоговорочно, у него есть ключи от продакшена. Если испортить любое раннее звено (подсунуть вредоносную библиотеку, украсть токен, подменить сборку), вредоносный код уедет к пользователям как «официальное» обновление. Работа строит модель угроз конвейера и систему, которая считает риск каждого этапа с учётом того, что заражение **передаётся по цепочке дальше**.

Проблема и цифры

- Sonatype: только за 2023 год в открытых репозиториях пакетов найдено более **245 тысяч** вредоносных компонентов.
- Громкие атаки на цепочку поставки: SolarWinds (2020), Codecov (2021), бэкдор в XZ Utils (CVE-2024-3094, 2024).
- Verizon DBIR 2024: около 15 % нарушений через цепочку поставки (вдвое больше, чем годом раньше). GitGuardian: утёкших в GitHub секретов 10 млн (2022) → 12,8 млн (2023). IBM: средняя стоимость утечки больше 4,8 млн долларов.

Что сделано в главе 1

- **1.1 Объект.** Конвейер как **цепочка доверия**: репозиторий → система сборки → зависимости → хранилище артефактов → хранилище секретов → среда развёртывания. Каждый следующий этап принимает результат предыдущего без проверки, поэтому компрометация раннего этапа наследуется всеми последующими. Таблица активов с классами OWASP. Модель нарушителя: внешний (через зависимости и сервисы) и внутренний (разработчик, администратор конвейера).
- **1.2 Недостаток CI/CD против уязвимости кода.** Недостаток конвейера порождается его организацией (права сборщика, срок жизни токенов, нет проверки подписи): у него нет CVE, нет CVSS, нет EPSS, поэтому в классический риск-анализ он «не попадает». Средства (Checkov, Snyk, GitGuardian, Trivy, SLSA/in-toto, Sigstore) фиксируют отдельные факты, но риск не считают и распространение по цепочке не учитывают. Примеры: Codecov и SolarWinds, где дефекта в коде не было.
- **1.3 Нормативка.** Приказы ФСТЭК № 17 и 239, методика оценки угроз, ГОСТ Р ИСО/МЭК 27001/27005, ГОСТ Р 56546; отраслевые NIST SP 800-218 (SSDF), NIST SP 800-204D, SLSA, OWASP Top 10 CI/CD. Нет измеримого критерия защищённости конвейера.
- **1.4 Источники.** Классы OWASP CICD-SEC-1…10 сопоставлены с техниками ATT&CK (T1195 цепочка поставки, T1059 выполнение кода, T1552 учётные данные в файлах, T1574/T1554 подмена, T1078 действительные учётки), плюс CAPEC и CWE.
- **1.5 База соответствий** «этап → недостаток → техника → ущерб» (полная версия в приложении А) и реестр секретов конвейера по области действия (приложение Б).
- **1.6 Классификация и дерево угроз.** Четыре системообразующие угрозы: **У1** компрометация зависимостей (typosquatting, dependency confusion), **У2** утечка секретов, **У3** подмена артефакта, **У4** злоупотребление правами сборщика (Poisoned Pipeline Execution). Риск R = P × U по

экспертным оценкам. Вывод: самые критичные риски сидят на **ранних** этапах (зависимости, секреты, сборка), потому что их последствия наследуются дальше.

- **1.7 Выводы по главе.**

Что предлагается в ВКР (6 задач)

1. База недостатков CI/CD по этапам цепочки доверия.
2. Инвентаризация секретов и методика оценки ущерба от их утечки (область действия, срок жизни, куда дают доступ).
3. Методика оценки доверия к артефакту (происхождение, воспроизводимость сборки, проверка подписи), шкала соответствия уровням SLSA.
4. Оценка критичности через адаптацию CVSS 4.0 с учётом зрелости цепочки поставки и прав сборщика.
5. Программа: ML-классификатор конвейеров по уровню риска, расчёт распространения компрометации, риск-ландшафт «этап → техника → риск».
6. Частные регламенты реагирования и меры защиты с расчётом снижения риска.

Особенность структуры

У Ширинкина шесть противоречий и шесть задач (у остальных по четыре), глава 1 называется «Модель угроз…» и содержит подраздел 1.7 «Выводы по первой главе», а также приложения А и Б. У Полыги, Ковалева и Перушкина используется общая схема с четырьмя задачами и главами 2–5.

Справочник А. Контейнеры и Kubernetes

Сначала: что такое контейнер

Виртуальная машина похожа на отдельный дом: у неё своя операционная система и своё ядро. **Контейнер** похож на квартиру в многоквартирном доме: у каждой квартиры свои стены, но фундамент (**ядро Linux**) общий для всех. Контейнер — это обычный процесс на сервере, которому ядро «надело шоры»:

- **namespaces (пространства имён ядра)** ограничивают, *что процесс видит*: свои процессы, свою сеть, свою файловую систему, своих пользователей;
- **cgroups (контрольные группы)** ограничивают, *сколько ресурсов* он может съесть: процессор, память.

Отсюда главный риск: стены программные. Если их ослабить настройками, взлом приложения превращается во взлом всего сервера.

Образ (image)

> Готовый «слепок» приложения со всеми библиотеками. Из одного образа можно запустить сколько угодно одинаковых контейнеров.

Реестр образов (registry)

> Склад образов (Docker Hub, Harbor и т. п.). Если подменить образ на складе, во все контейнеры попадёт чужой код.

Docker, containerd, CRI-O

> Программы, которые собирают и запускают контейнеры. containerd и CRI-O называют **средой исполнения контейнеров** (container runtime).

Микросервисы

> Способ строить систему из множества маленьких сервисов (каждый в своём контейнере) вместо одной большой программы.

Kubernetes: «диспетчер» для тысяч контейнеров

Когда контейнеров сотни, вручную раскладывать их по серверам невозможно. **Kubernetes** (сокращённо K8s) — система **оркестрации**: ты описываешь *желаемое состояние* («хочу 3 копии веб-приложения из такого-то образа, с таким-то паролем от БД»), а Kubernetes сам выбирает серверы, запускает контейнеры, перезапускает упавшие и добавляет копии при нагрузке. Описание пишется в YAML-файле, который называется **манифест**.

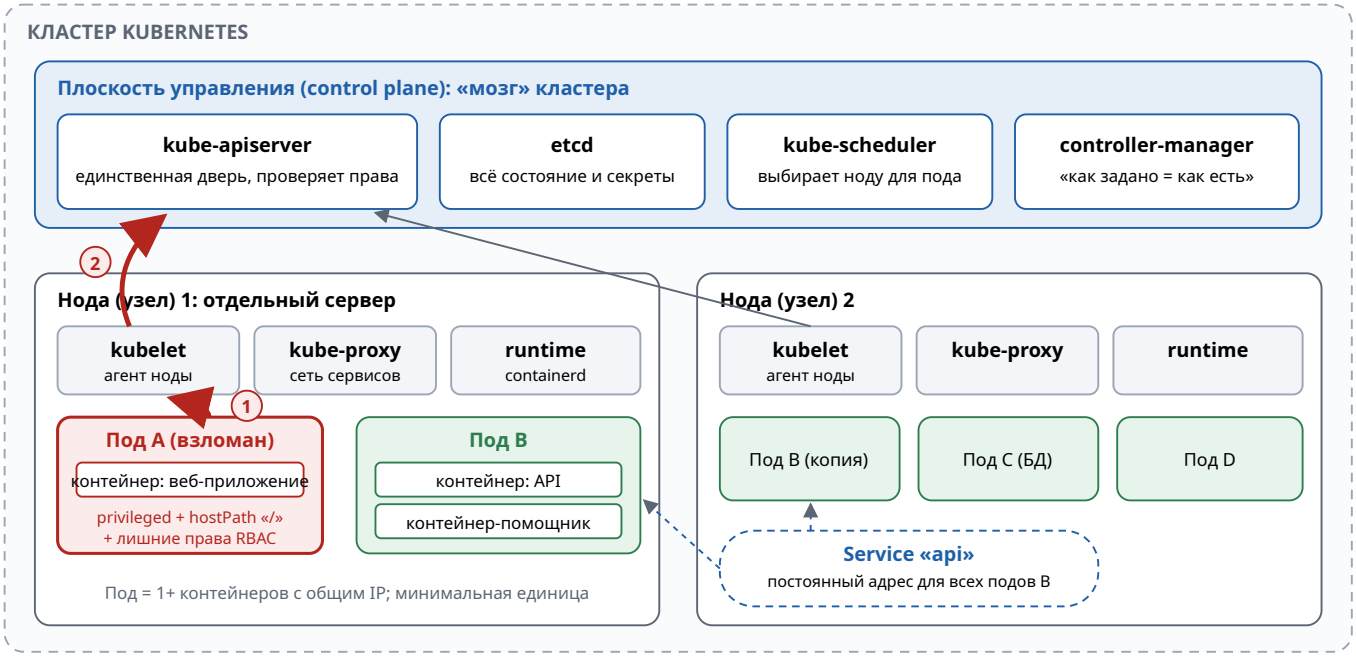


Рисунок 2. Как устроен кластер и как из одного взломанного пода захватывают всё (красные стрелки: 1 — через опасные настройки пода нарушитель выходит на ноду и получает её учётные данные; 2 — с правами ноды или сервисного аккаунта идёт в kube-apiserver, а значит к управлению всем кластером)

Из чего состоит кластер

Кластер

> Группа серверов, которыми управляет один Kubernetes.

Нода (узел, node)

> Один сервер (железный или виртуальный) в кластере. На *рабочих* нодах реально крутятся контейнеры. Если захвачена нода, захвачены все поды на ней и их секреты.

Плоскость управления (control plane)

> «Мозг» кластера, обычно на отдельных нодах. Состоит из четырёх частей:
>
> - **kube-apiserver** — единственная «дверь». Любая команда (запустить, удалить, прочитать секрет) идёт через него, и он проверяет: кто ты (аутентификация), можно ли тебе (авторизация, RBAC), допустим ли такой объект (admission). Захват = полное управление кластером.
> - **etcd** — база данных, где хранится всё состояние кластера, включая **секреты** (пароли, токены). Утечка etcd = утечка всех секретов.
> - **kube-scheduler** — решает, на какую ноду поставить новый под.
> - **kube-controller-manager** — набор «надсмотрщиков»: если задано 3 копии, а работает 2, он создаст третью.

kubelet

> Агент на каждой ноде: получает от apiserver задание «запусти такой-то под» и командует среде исполнения. Если kubelet пускает анонимные запросы, через него можно управлять контейнерами ноды.

kube-proxy

> Настраивает сетевые правила на ноде, чтобы запросы к Service доходили до нужных подов.

Объекты, которые описываются манифестами

Под (pod)

> Минимальная единица, которую запускает Kubernetes: «обёртка» вокруг одного или нескольких контейнеров, которые делят один IP-адрес и могут делить диски. Поды «смертны»: упал, и вместо него создаётся новый, с новым IP. В работе Полыги под (рабочая нагрузка) и есть единица, для которой считается риск.

Рабочая нагрузка (workload), Deployment

> Общее название для того, что запускает поды. **Deployment** говорит «держи N одинаковых подов и обновляй их плавно».

Service (сервис)

> Постоянное имя и адрес для группы одинаковых подов. Поды меняются, а адрес сервиса остаётся, и запросы балансируются между живыми подами. Это не «служба Windows», а именно сетевая точка входа.

Namespace (пространство имён Kubernetes)

> Логическая «папка» внутри кластера для разделения команд и проектов. Не путать с namespaces ядра Linux из раздела про контейнеры.

Secret, ConfigMap

> Объекты для хранения паролей и токенов (Secret) и обычных настроек (ConfigMap), которые подставляются в поды.

ServiceAccount (сервисный аккаунт)

> «Учётная запись» пода. Под ходит в kube-apiserver от её имени. Если у аккаунта лишние права, взломщик пода их получает.

RBAC (ролевое управление доступом)

> **Role/ClusterRole** описывает, *что можно* («читать секреты», «создавать поды»), **RoleBinding/ClusterRoleBinding** — *кому* это разрешено. «Избыточная привязка роли» значит, что поду выдали больше, чем ему нужно (по статистике у 98 % аккаунтов есть неиспользуемые права).

Опасные настройки, о которых идёт речь у Полыги

Настройка	Что значит простыми словами	Чем опасна
`privileged: true` (привилегированный под)	С контейнера сняты почти все ограничения ядра	Фактически root на ноде, выход из контейнера тривиален
`hostPath` (монтирование каталога узла)	Папка сервера (например, `/` или `/etc`) видна внутри контейнера	Можно читать и менять файлы ноды, подкладывать ключи и задания
`hostNetwork`, `hostPID`	Контейнер использует сеть или процессы самой ноды	Видит чужие процессы и трафик, обходит сетевую изоляцию
Смонтирован сокет среды исполнения (`docker.sock`, `containerd.sock`)	Контейнер может «разговаривать» с программой, запускающей контейнеры	Можно запустить свой привилегированный контейнер = захват ноды
Запуск от root, без ограничений ресурсов	Процесс внутри контейнера root; может занять всю память и CPU	Облегчает побег; атака на доступность соседей

Анонимный доступ к apiserver или kubelet	Можно обращаться без логина	Управление кластером или нодой без взлома

Admission controller, Pod Security Standards

> «Охранник на входе» в kube-apiserver: не пускает манифесты с опасными настройками. Pod Security Standards задают три профиля: privileged (всё можно), **baseline** (запрещено явно опасное), **restricted** (строго).

Граница доверия

> Рубеж, при пересечении которого у нарушителя резко растут возможности: контейнер → нода → плоскость управления; одно пространство имён → другое; кластер → внешний реестр.

Инструменты проверки Kubernetes

kube-bench

> Проверяет настройки компонентов кластера по чек-листу CIS Kubernetes Benchmark. Результат: PASS / FAIL / WARN по каждому пункту.

Kubescape

> Проверяет объекты кластера и манифесты сразу по нескольким профилям (CIS, NSA/CISA), связывает найденное с техниками MITRE ATT&CK, выдаёт процент соответствия.

Trivy

> Сканер: ищет известные уязвимости (CVE) в образах и ошибки в манифестах. Выдаёт список с уровнями опасности и CVSS.

Falco

> «Сигнализация» во время работы: следит за системными вызовами контейнеров и поднимает тревогу при подозрительном (например, внутри контейнера запустили shell). Работает по факту, а не заранее.

Справочник Б. Сервер на Linux и сканеры (к работе Перушкина)

Сервис (служба, демон)

> Программа, которая постоянно работает в фоне и обслуживает запросы: веб-сервер (nginx, Apache), СУБД (PostgreSQL), Redis, SSH-сервер. Управляется **systemd**.

«Слушает интерфейс» (127.0.0.1 против 0.0.0.0)

> Сервис ждёт подключений на сетевом адресе. `127.0.0.1` (localhost) — подключиться можно только с самого сервера. `0.0.0.0` — со всех сетевых карт, то есть потенциально из интернета. Одна строка в конфиге решает, доступна ли уязвимость снаружи.

SSH, sshd

> Протокол и сервер удалённого входа на сервер с шифрованием. Ключевые параметры `sshd_config`: `PermitRootLogin` (можно ли входить сразу root), `PasswordAuthentication` (вход по паролю или только по ключу), `MaxAuthTries` (сколько попыток).

root (суперпользователь)

> Учётная запись с неограниченными правами на сервере. «Повышение привилегий» = путь от обычного пользователя до root.

sudo

> Команда «выполнить от root». Правила в `/etc/sudoers`. Строка вида `user ALL=(ALL) NOPASSWD: ALL` — «безусловное повышение полномочий»: кто захватил user, тот сразу root.

SUID-бит

> Отметка на файле «запускать с правами владельца». Если владелец root, любой пользователь запускает программу с правами root. Так работает pkexec, и именно поэтому уязвимость в нём (CVE-2021-4034, PwnKit) даёт root.

PAM в Linux (Linux-PAM)

> Pluggable Authentication Modules — «конструктор» проверки входа: пароль, блокировка после N неудач, второй фактор. *Не путать* с PAM у Ковалева (Privileged Access Management).

Мандатный контроль доступа: SELinux, AppArmor, мандатный контроль Astra Linux

> Дополнительный слой, который ограничивает даже root строгой политикой («этот процесс может трогать только эти файлы»). Если его отключить (частая «ошибка для удобства»), взлом сервиса даёт больше.

cron, таймеры systemd

> Планировщик: запускает команды по расписанию. Задание, которое выполняется от root и лежит в папке, доступной на запись обычному пользователю, — классический путь к root.

Журналирование: journald, rsyslog, auditd

> Запись событий. Если журналы не пишутся, хранятся только локально или их может стереть взломщик, расследовать инцидент невозможно.

Пакетный менеджер (apt, dnf) и репозитории

> Установка и обновление программ. Важно, чтобы пакеты брались из доверенных источников с проверкой цифровой подписи.

Redis

> Очень быстрая база данных в памяти. Исторически часто ставится без пароля. Если при этом слушает 0.0.0.0, любой может отправлять ему команды; отсюда пример с CVE-2022-0543.

Astra Linux

> Российский дистрибутив Linux с сертификатами ФСТЭК, основной кандидат для импортозамещения в госсекторе и КИИ.

Ошибка конфигурирования, конфигурационный дрейф

> Ошибка конфигурирования — допустимое, но опасное значение параметра (не баг в коде). Дрейф — постепенное расползание реальных настроек от утверждённых (ручные правки, откаты, восстановление из бэкапа). Из-за дрейфа разовая проверка быстро устаревает.

Компенсирующая мера

> Защита вместо обновления, когда патч поставить нельзя: закрыть порт, отключить модуль, изменить параметр (пример с `LoginGraceTime` для CVE-2024-6387 «regreSSHion» в OpenSSH).

Сканеры и средства проверки

Средство	Что делает
SCAP, OVAL	Стандарты машиночитаемых проверок. OVAL-описание — это «рецепт»: какой файл или параметр посмотреть и какое значение считать плохим. SCAP — общий набор таких стандартов.
OpenSCAP	Открытая программа, выполняющая SCAP/OVAL-проверки: и соответствие профилю настроек, и наличие известных уязвимостей.
Lynis	Аудитор Linux: прогоняет встроенные проверки, даёт советы и общий «индекс защищённости».
CIS Benchmarks, CIS-CAT	CIS Benchmarks — подробные чек-листы безопасной настройки под каждую ОС и продукт. CIS-CAT — программа, проверяющая по ним и выдающая процент выполненных требований.
DISA STIG	Чек-листы безопасной настройки Минобороны США, у каждого требования категория важности I/II/III.
Wazuh	Открытая платформа безопасности (SIEM/XDR): агенты на серверах, проверка конфигурации и уязвимостей по списку пакетов.
ScanOVAL	Бесплатный сканер ФСТЭК: сверяет установленные пакеты с уязвимостями из БДУ ФСТЭК.
Nessus, MaxPatrol VM	Коммерческие сканеры и системы управления уязвимостями (зарубежный и отечественный от Positive Technologies): инвентаризация, поиск CVE, приоритизация.
MulVAL	Исследовательский логический анализатор: по результатам сканирования и описанию сети строит граф атак (какие привилегии может получить нарушитель и как).

Справочник В. Корпоративный доступ (к работе Ковалева)

Субъект доступа, учётная запись

> Субъект — кто обращается (человек или программа). Учётная запись — его «пропуск» в системе: логин, пароль или ключ, группы, роли. Система доверяет пропуску, а не человеку: кто предъявил, тот и получил все права.

Служба каталога (Active Directory, FreeIPA, ALD Pro)

> Центральный «справочник» организации: все пользователи, группы, компьютеры, подразделения и их права. Источник истины при решении «пускать или нет». Active Directory — самая распространённая (Microsoft); FreeIPA и ALD Pro — Linux-аналоги.

LDAP

> Протокол, по которому программы читают и ищут записи в каталоге. Любой сотрудник штатными запросами может выгрузить структуру групп, поэтому «перечисление каталога» даёт нарушителю карту путей к администским правам.

Группы и вложенность групп

> Права выдаются группе, пользователи включаются в группы. Группа может входить в другую группу, и права передаются по цепочке. Итоговые права становятся неочевидными.

Аутентификация и авторизация

> Аутентификация — доказать, что ты это ты (пароль, ключ, код). Авторизация — решить, что тебе можно.

Протоколы входа и атаки на них

Kerberos

> Основной протокол входа в домене. Работает на «билетах», как проходная: один раз подтвердил личность у центра выдачи (KDC) и получил «общий пропуск» (TGT), по нему получаешь «билеты» к конкретным сервисам и больше пароль не вводишь. Ключ, которым подписываются общие пропуска, принадлежит служебной учётке `krbtgt`.

NTLM

> Старый протокол Microsoft, в котором вместо пароля передаётся производная от него величина (хэш).

Pass-the-Hash

> Украсть хэш пароля и входить им напрямую, не зная самого пароля (возможно из-за NTLM).

Kerberoasting

> Любой пользователь может запросить билет к сервису; билет зашифрован паролем сервисной учётки. Его уносят и подбирают пароль у себя, без попыток входа, то есть без следов в журналах. Поэтому слабые пароли сервисных учёток опасны.

Golden Ticket («золотой билет»)

> Если украден ключ `krbtgt`, можно самому «напечатать» пропуск с любыми правами и любым сроком действия. Это полная и долгая власть над доменом.

SSO (единый вход), SAML, OpenID Connect

> Вошёл один раз и получил доступ ко всем связанным системам. Удобно, но украденная сессия открывает сразу всё.

MFA (многофакторная аутентификация)

> Несколько разных доказательств: знаю (пароль), имею (телефон, токен), являюсь (отпечаток).

AAL1 / AAL2 / AAL3 (NIST SP 800-63B)

> Уровни надёжности входа. **AAL1**: один фактор (пароль). **AAL2**: два фактора (пароль и код из SMS или приложения, push). **AAL3**: два фактора, один из них аппаратный криптоключ, устойчивый к перехвату (FIDO2/USB-ключ, смарт-карта).

Фишинг в реальном времени (AiTM)

> Поддельная страница входа, которая сразу пересылает введённые пароль и код на настоящий сайт. Обычный MFA с кодом от этого не спасает, аппаратный ключ спасает.

Утомление подтверждениями (MFA fatigue, push bombing)

> Зная пароль, нарушитель заваливает сотрудника push-запросами «Это вы входите?», пока тот не нажмёт «Да».

Модели доступа и классы средств

DAC, MAC, RBAC, ABAC

> **DAC** (дискреционная): права раздаёт владелец объекта. **MAC** (мандатная): доступ по меткам секретности. **RBAC** (ролевая): права даются роли («бухгалтер»), роль — человеку. **ABAC** (атрибутная): решение вычисляется по атрибутам (отдел, время, место). В RBAC видно, чем человек отличается от коллег той же должности; на этом у Ковалева построен поиск аномалий.

Принцип минимальных привилегий, накопление полномочий

> Правило: давать ровно столько прав, сколько нужно для работы. Накопление (privilege creep) — нарушение этого правила со временем: права за прошлые должности и проекты остаются.

IdM / IGA

> Identity Management / Identity Governance and Administration. Автоматизируют жизненный цикл учётки (принят, переведён, уволен), заявки и согласование прав, **аттестацию** (руководитель периодически подтверждает права подчинённых), контроль **разделения полномочий** (SoD: один человек не должен и создавать, и оплачивать счёт).

PAM (Privileged Access Management)

> Сейф паролей администраторов, выдача админских прав на время по заявке, запись админских сессий на видео. Работает только с «привилегированными» учётками.

Аудит службы каталога

> Разовое обследование AD на опасные сочетания прав, пути повышения привилегий, вечные пароли админов, устаревшие протоколы (например, инструменты вроде PingCastle или BloodHound).

SIEM

> Система сбора и корреляции событий безопасности со всей инфраструктуры. Реагирует на то, что уже произошло.

UEBA

> Поведенческий анализ пользователей: учится, как человек обычно работает, и сигналит об отклонениях (вход ночью, обращение к чужим ресурсам). Тихое накопление прав не замечает, потому что поведение не меняется.

Машинное обучение у Ковалева

Обучение с учителем и без учителя

> С учителем: модели показывают примеры с правильными ответами («эта учётка опасна»). Без учителя: ответов нет, модель сама ищет структуру и выбросы. Размеченных данных «у кого лишние права» в организациях нет, поэтому выбран второй вариант.

Изолирующий лес (Isolation Forest)

> Алгоритм поиска аномалий. Строит много случайных деревьев, каждое режет данные случайными разрезами по признакам. Обычную точку, окружённую похожими, приходится резать долго; аномалию отрезают за пару разрезов. Чем меньше разрезов понадобилось, тем аномальнее объект. Пример: у всех бухгалтеров по 5–7 групп, у одного 40, и он изолируется почти сразу. Плюс: видно, по какому признаку он выделился, то есть результат можно объяснить.

Справочник Г. DevOps и CI/CD (к работе Ширинкина)

DevOps

> Подход, при котором разработка и эксплуатация работают как единый автоматизированный процесс: код быстро и часто уезжает в продакшен (зрелые команды выкладывают по несколько раз в день).

CI (Continuous Integration, непрерывная интеграция)

> Каждое изменение кода автоматически собирается и прогоняется тестами.

CD (Continuous Delivery / Deployment, непрерывная поставка / развёртывание)

> Собранное автоматически доставляется и разворачивается на серверах.

Конвейер (pipeline), задание (job), исполнитель (runner)

> Конвейер — последовательность шагов «собрать → проверить → упаковать → выложить». Каждый шаг — задание. Исполнитель (runner, агент) — машина или контейнер, которая выполняет задания; у неё обычно много прав и доступ к секретам. Примеры систем: GitLab CI, GitHub Actions, Jenkins.

Репозиторий, Git, ветка, защита веток

> Репозиторий — хранилище кода с историей всех изменений (Git). Ветка — параллельная линия изменений. Защита веток — правило «в main нельзя залить без ревью и проверок». Без неё любой разработчик может протолкнуть что угодно.

Pipeline-as-code, GitOps

> Описание конвейера хранится файлом прямо в репозитории (например, `.gitlab-ci.yml`), а при GitOps и состояние инфраструктуры тоже. Следствие: кто может менять репозиторий, тот может менять и сам конвейер.

Зависимость (транзитивная зависимость)

> Чужая библиотека, которую код подключает из npm, PyPI, Maven. Транзитивная — зависимость зависимости: её никто сознательно не выбирал.

Артефакт, хранилище артефактов

> Результат сборки: исполняемый файл, пакет, образ контейнера. Хранится в хранилище (Nexus, Artifactory) или реестре образов, оттуда уходит на развёртывание.

Секрет, хранилище секретов

> Секрет — пароль, токен, ключ, по которому конвейер ходит в репозиторий, реестр и продакшен. Хранилище секретов (например, HashiCorp Vault) выдаёт их заданиям. «Долгоживущий токен» годами не меняется и после утечки годами работает; современная практика — краткоживущие токены, выдаваемые на один запуск.



Рисунок 3. Цепочка доверия конвейера и четыре угрозы Ширинкина: У1 зависимости, У2 секреты, У3 подмена артефакта, У4 права сборщика

Атаки на конвейер

Атака на цепочку поставки (supply chain attack)

> Взламывают не жертву, а то, чем она пользуется: библиотеку, инструмент, сервис сборки. Вредоносный код приходит к тысячам клиентов как официальное обновление.

Typosquatting

> Публикация вредоносного пакета с похожим именем (`reqeusts` вместо `requests`), в расчёте на опечатку.

Dependency confusion (путаница зависимостей)

> У компании есть внутренний пакет `corp-utils`. Нарушитель публикует в публичном реестре пакет с тем же именем и большей версией, и сборщик по ошибке берёт публичный.

Poisoned Pipeline Execution (PPE, отравление конвейера)

> Нарушитель меняет код или описание конвейера (например, в pull request), и раннер исполняет его со своими правами и доступом к секретам.

SolarWinds (2020)

> В систему сборки продукта Orion внедрили закладку. Собранное и *официально подписанное* обновление разошлось по клиентам, включая госструктуры США.

Codecov (2021)

> Изменили скрипт загрузки (Bash Uploader), который запускался в конвейерах множества компаний; несколько месяцев он собирал секреты из их сред сборки.

XZ Utils (2024, CVE-2024-3094)

> Злоумышленник годами вёл себя как добросовестный соавтор популярной библиотеки сжатия, стал сопровождающим и внедрил бэкдор, который через цепочку зависимостей затрагивал SSH-сервер в ряде дистрибутивов. Обнаружили случайно перед попаданием в стабильные релизы.

Защита и инструменты

Подпись артефакта, provenance (происхождение)

> Подпись доказывает, что артефакт не меняли после сборки. Provenance — «паспорт» сборки: из какого коммита, каким сборщиком, с какими зависимостями собрано.

SLSA

> Supply-chain Levels for Software Artifacts — лестница уровней зрелости защиты цепочки поставки: от «есть описание сборки» до «сборка изолирована, провенанс подписан и не подделывается».

in-toto, Sigstore / cosign

> in-toto — фреймворк, подтверждающий, что каждый шаг конвейера сделан тем, кем положено. Sigstore/cosign — инструменты подписи и проверки артефактов и образов.

Воспроизводимая сборка

> Из одного и того же исходного кода всегда получается бит-в-бит одинаковый результат, значит можно доказать, что артефакт соответствует проверенному коду.

SAST, SCA, secret scanning, IaC-сканирование

> SAST ищет ошибки в собственном исходном коде. SCA ищет известные уязвимости в зависимостях. Secret scanning ищет пароли и токены в коде и истории. IaC-сканирование проверяет описания инфраструктуры (Terraform, Kubernetes-манифесты).

Checkov, Snyk, GitGuardian, Trivy

> Checkov — проверка IaC-конфигураций на соответствие политикам. Snyk — уязвимости в зависимостях и образах. GitGuardian — поиск утёкших секретов. Trivy — универсальный сканер уязвимостей и ошибок конфигурации.

OWASP Top 10 CI/CD Security Risks

> Список десяти главных рисков конвейеров (CICD-SEC-1…10): контроль потока изменений, доступ, зависимости, отравление конвейера, права раннеров, гигиена секретов, небезопасная конфигурация, сторонние сервисы, целостность артефактов, журналирование.

Справочник Д. Риск-анализ: базы, калькуляторы, модели

Базы знаний об уязвимостях и атаках

Что	Что это	Пример
CVE	Международный реестр известных уязвимостей: каждой дыре в конкретном продукте даётся уникальный номер.	CVE-2021-4034 (pkexec)
NVD	Национальная база уязвимостей США: к каждой CVE добавляет описание и оценку CVSS.	nvd.nist.gov
БДУ ФСТЭК	Банк данных угроз ФСТЭК России: отечественный реестр уязвимостей (BDU:...) и перечень угроз (УБИ....). Для ГИС и КИИ основной источник.	BDU:2022-00488
CWE	Классификатор *типов* слабостей (не конкретных дыр).	CWE-798 «пароль зашит в код»
CAPEC	Каталог типовых *шаблонов атак*.	CAPEC-437 атака через цепочку поставки
MITRE ATT&CK	Матрица реальных действий хакеров. **Тактика** = зачем шаг (первоначальный доступ, закрепление, повышение привилегий, доступ к учётным данным...). **Техника** = как именно. Есть отдельная матрица для контейнеров.	T1078 «использование действительных учётных записей»
CISA KEV	Каталог уязвимостей, которые *точно* эксплуатируются в реальных атаках (ведёт агентство CISA, США).	CVE-2022-0543 (Redis)

Калькуляторы опасности

CVSS 3.1

> Шкала опасности уязвимости от 0 до 10 (0,1–3,9 низкая; 4–6,9 средняя; 7–8,9 высокая; 9–10 критическая). Считается по метрикам: **базовые** (откуда атакуют: сеть или локально; сложность; нужны ли права; нужен ли пользователь; что страдает: конфиденциальность, целостность, доступность), **временные** (есть ли эксплойт и патч), **средовые** (уточнение под конкретную систему). На практике публикуют только базовую оценку, «в вакууме».

CVSS 4.0

> Новая версия: временные метрики заменены **метриками угрозы** (зрелость эксплойта), отдельно учитываются последствия для уязвимой системы и для **последующих систем** (важно для «побега» из контейнера), добавлены дополнительные метрики (автоматизируемость атаки и др.). Средовые метрики позволяют пересчитать оценку по реальным настройкам (сервис на localhost: вектор «локальный», а не «сетевой»). Но как именно выводить их из конфигов, стандарт не говорит, и на этом строится часть работы Перушкина.

EPSS

> Прогноз вероятности (от 0 до 1), что уязвимость начнут эксплуатировать в ближайшие 30 дней. CVSS отвечает «насколько больно», EPSS отвечает «насколько вероятно». Работает только для CVE.

Почему это общая проблема всех четырёх работ

CVSS, EPSS, KEV работают «по номеру CVE». У лишней роли RBAC, у разросшейся учётки, у Redis на 0.0.0.0, у вечного токена конвейера номера CVE нет, значит нет и оценки. Каждая работа предлагает способ получить для таких недостатков сопоставимые числа (перенос метрик CVSS, ML-модели, FAIR).

Модели риска

Мера риска R = P · U

> Вероятность наступления ущерба, умноженная на величину ущерба (определение из методики кафедры).

FAIR (Factor Analysis of Information Risk)

> Модель, которая раскладывает риск на множители, чтобы каждый можно было оценить отдельно:
>
> - **Риск** = частота событий потерь (**LEF**) × величина потерь (**LM**);
> - **LEF** = частота попыток атаки (**TEF**) × доля успешных попыток (**Vulnerability**);
> - потери бывают первичные (свои: простой, восстановление) и вторичные (штрафы, иски, репутация).
>
> Каждый множитель задаётся не числом, а диапазоном («от 2 до 10 атак в год»), и методом **Монте-Карло** (тысячи случайных прогонов) получается распределение годовых потерь, например «с вероятностью 90 % потери не больше 3 млн руб.». Принята как открытый стандарт The Open Group.

SLE, ARO, ALE

> **SLE** — ущерб от одного инцидента; **ARO** — сколько раз в год ожидается; **ALE = SLE × ARO** — ожидаемые потери в год. Пример: инцидент стоит 2 млн руб. и случается раз в два года (ARO = 0,5), тогда ALE = 1 млн руб. в год.

ROSI (возврат инвестиций в безопасность)

> **ROSI = $(ALE_1 - ALE_2 - C) / C$**, где ALE_1 — потери до меры (первичный риск), ALE_2 — после (остаточный), C — стоимость меры.
> *Условный пример:* ALE_1 = 10 млн, ALE_2 = 2 млн, мера стоит 1 млн. ROSI = (10 − 2 − 1) / 1 = 7, то есть каждый вложенный рубль экономит 7 рублей сверх затрат. Если ROSI < 0, мера не окупается.

Первичный и остаточный риск

> Первичный — риск как есть, до мер. Остаточный — что осталось после внедрения мер. Их разница и есть эффект мер.

OCTAVE Allegro, NIST SP 800-30, ГОСТ Р ИСО/МЭК 27005, ГОСТ Р ИСО 31000

> Классические методики и стандарты менеджмента риска: задают процесс, но оценки в них в основном экспертные и качественные («высокий / средний / низкий»).

Деревья, графы, ландшафты

Дерево атак / дерево угроз

> Корень — цель нарушителя или главная угроза; ветви — способы её достичь, соединённые условиями «И» (нужно всё) и «ИЛИ» (достаточно одного). Удобно для полноты перечня, но не отражает конкретную инфраструктуру.

Граф атак

> Узлы — состояния («нарушитель имеет shell в поде A», «имеет root на ноде 1»), рёбра — действия, переводящие из одного состояния в другое. Путь от входа до ценного актива = сценарий атаки.

> Если у рёбер есть вероятности, можно посчитать вероятность всего пути.

Риск-ландшафт

> «Карта рисков» объекта: все риски, упорядоченные по величине и привязанные к месту (учётке, поду, этапу конвейера). Показывает, что чинить первым.

Проактивный vs реактивный подход

> Реактивный: реагируем на случившийся инцидент (SIEM, Falco, UEBA). Проактивный: оцениваем риск заранее по состоянию системы и устраняем причины до атаки. Все четыре работы про проактивный.

Масштабность и профильность мер

> Термины методики кафедры для классификации мер защиты: насколько широко мера действует (на один объект или на всю систему) и насколько она специфична для данного класса угроз.

Справочник E. Нормативка, которая упоминается в работах

Документ	О чём	У кого
149-ФЗ, 152-ФЗ, 187-ФЗ	Об информации и защите информации; о персональных данных; о безопасности КИИ	Ковалев, Перушкин
Указ Президента № 166 (30.03.2022)	Технологическая независимость КИИ, переход на отечественное ПО	Перушкин
Приказ ФСТЭК № 17	Меры защиты в государственных информационных системах (ГИС). Группы мер: **ИАФ** (идентификация и аутентификация), **УПД** (управление доступом), **РСБ** (регистрация событий), **ОЦЛ** (целостность), **УКФ** (управление конфигурацией) и др.	все
Приказ ФСТЭК № 21	Меры защиты персональных данных в ИСПДн	Полыга, Ковалев, Перушкин
Приказ ФСТЭК № 239	Безопасность значимых объектов КИИ	все
Методика оценки угроз ФСТЭК (2021)	Как определять актуальные угрозы: нарушитель, способ, объект воздействия, последствия	все
Методический документ ФСТЭК по безопасной настройке Linux; методика оценки критичности уязвимостей; руководство по управлению уязвимостями	Конкретные требования к параметрам Linux; как приоритизировать уязвимости с учётом своей системы; процесс устранения, включая компенсирующие меры	Перушкин
ГОСТ Р 50922	Основные термины защиты информации	Ковалев, Перушкин
ГОСТ Р 56545, 56546	Правила описания и классификация уязвимостей. Важно: выделяют **уязвимости конфигурации** как отдельный класс, то есть нормативно признают, что плохая настройка тоже уязвимость	все
ГОСТ Р 56938	Защита информации при использовании виртуализации	Полыга
ГОСТ Р 58833	Идентификация и аутентификация, уровни доверия	Ковалев
ГОСТ Р 57580.1	Защита информации финансовых организаций	Ковалев
ГОСТ Р ИСО/МЭК 27001, 27005; ГОСТ Р ИСО 31000; ГОСТ Р 58771	Системы менеджмента ИБ; менеджмент риска ИБ; общие принципы риск-менеджмента; технологии оценки риска	все
NIST SP 800-190	Безопасность контейнерных приложений: риски образов, реестров, оркестратора, контейнеров, ОС узла	Полыга
CIS Kubernetes Benchmark, Pod Security Standards	Чек-лист настроек кластера; профили допустимых настроек подов	Полыга
NIST SP 800-63B	Цифровая идентичность, уровни AAL	Ковалев
NIST SP 800-53, 800-30	Каталог мер защиты; руководство по оценке рисков	Ковалев, Перушкин

NIST SP 800-218 (SSDF), 800-204D	Практики безопасной разработки; безопасность цепочки поставки в CI/CD	Ширинкин
CIS Benchmarks, DISA STIG	Чек-листы безопасной настройки ОС и продуктов	Перушкин
OWASP Top 10 CI/CD, SLSA	Главные риски конвейеров; уровни зрелости цепочки поставки	Ширинкин

Как пользоваться этим документом

Перед чтением чужой НИРС прочитайте раздел 1 и краткую рамку автора: этого достаточно, чтобы понимать логику каждого абзаца. Если встретилось незнакомое слово, ищите его в справочнике, соответствующем теме автора (А для Полыги, В для Ковалева, Б для Перушкина, Г для Ширинкина); общие термины риск-анализа собраны в справочнике Д.